Machine Learning — Challenge 5
Unsupervised Learning: Clustering Open Government Data
`K-Means`, `DBSCAN`, and `Hierarchical Clustering`

`Prof. Carlos Andrés Sierra, M.Sc.`
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

This challenge introduces `unsupervised clustering` for students in the Machine Learning course using real-world open government data from U.S. sources. Each group works on the same domain assigned in **Challenge 2**, allowing a natural comparison of paradigms: in Challenge 2 students extracted signal from limited labels; here there are **no labels at all** during training. The objective is to discover latent structure in the data through three complementary clustering algorithms: `K-Means`, `DBSCAN`, and `Hierarchical (Agglomerative) Clustering`.

Students will design a rigorous comparative study, evaluate each algorithm with standardised internal metrics, interpret the discovered clusters in domain terms, and deliver a concise scientific report in IEEE format.

Learning objectives

Students will learn to:

- *Preprocess* real-world tabular data for clustering: normalisation, missing value handling, feature selection, and dimensionality reduction.
- *Implement and configure* three clustering algorithms (`K-Means`, `DBSCAN`, `Hierarchical`) using `scikit-learn`.
- *Select hyperparameters* systematically: number of clusters k, density radius ε and minimum samples for `DBSCAN`, and linkage criterion for `Hierarchical clustering`.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

- *Evaluate* cluster quality with internal validation metrics (Silhouette Score, Davies–Bouldin Index, Calinski–Harabasz Index) and interpret the results.
- *Visualise* high-dimensional clusters using `PCA` and optionally `t-SNE` or `UMAP`.
- *Interpret* discovered clusters in domain context and discuss their practical meaning.
- *Compare* algorithm behaviour and identify which method is best suited to the chosen dataset, justifying the conclusion with empirical evidence.

Challenge objective

Each group will:

1. Select a suitable dataset from their assigned domain (same domain as Challenge 2).
2. Perform exploratory data analysis (`EDA`), preprocessing, and feature engineering.
3. Apply all three required clustering algorithms to the same preprocessed dataset.
4. Conduct a *hyperparameter sensitivity study* for each algorithm.
5. Evaluate and compare results using standardised metrics and visualisations.
6. Produce a scientific report discussing the structure discovered and what it reveals about the real-world domain.

Domain assignments (one per group, same as Challenge 2)

Each group (1–8) continues working on the domain assigned in Challenge 2. The table below re-states each assignment and highlights the **clustering-specific opportunity** within that domain.

1. **Group 1 — Transportation & Traffic Safety.** NHTSA FARS or BTS crash records. Cluster accident profiles (severity, road type, time-of-day, weather conditions) to discover distinct risk archetypes. Does `DBSCAN` reveal spatial hotspot clusters that `K-Means` misses?
2. **Group 2 — Environment & Air Quality.** EPA AQS sensor measurements. Cluster monitoring stations or time windows by pollutant co-occurrence patterns. Explore whether geographic proximity aligns with cluster membership.
3. **Group 3 — Public Health & Epidemiology.** CDC WONDER county-level mortality or BRFSS behavioural survey data. Cluster counties or respondents by health behaviour / outcome profiles. Identify vulnerable demographic archetypes without using any outcome label.
4. **Group 4 — Education Systems.** NCES Common Core of Data. Cluster schools or districts by resource, demographic, and outcome features. Investigate whether the discovered groups reflect known urban/rural or socioeconomic divisions.

5. **Group 5 — Economics & Socioeconomic Data.** ACS microdata from U.S. Census. Cluster census tracts or respondents by income, employment, and household structure. Evaluate cluster interpretability and compare with Census-defined categories.

6. **Group 6 — Housing & Urban Development.** HUD or local assessor datasets. Cluster properties or neighbourhoods by structural and demographic features. Study whether `DBSCAN` identifies spatial outliers (anomalous properties) that other methods absorb into clusters.

7. **Group 7 — Crime & Public Safety.** FBI Crime Data Explorer or city open-data portals (Chicago, NYC). Cluster incident types, spatial zones, or time periods. Evaluate whether temporal and spatial features generate consistent cluster structures across algorithms.

8. **Group 8 — Energy & Utilities.** EIA consumption time-series or state utility datasets. Cluster customers, regions, or time intervals by consumption patterns. Explore whether `DBSCAN` identifies anomalous-consumption outliers as noise distinct from normal-consumption clusters.

Required techniques

All three algorithms are **mandatory**. Each must be applied to the same preprocessed feature matrix so results are directly comparable.

`K-Means`

`K-Means` partitions data into k non-overlapping clusters by iteratively assigning points to the nearest centroid and updating centroids. It assumes roughly *spherical, equal-size* clusters and is sensitive to *outliers* and *feature scale*.

Key parameters:

- **`n_clusters`** (k): the number of clusters to form. Must be chosen a priori. Use the *Elbow method* (inertia vs. k) and the *Silhouette score* to guide selection across a range such as $k \in \{2, 3, \ldots, 12\}$.
- **`init`**: **`'k-means++'`** (default) for better centroid initialisation; compare with **`'random'`** to assess stability.
- **`n_init`**: run at least 10 restarts to mitigate local-optima sensitivity.

General usage pattern:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10,
                random_state=42)
labels = kmeans.fit_predict(X_scaled)
```

DBSCAN

`DBSCAN` (Density-Based Spatial Clustering of Applications with Noise) discovers clusters of *arbitrary shape* and explicitly marks low-density points as *noise* (label -1). It does not require specifying k but is sensitive to the density parameters.

Key parameters:

- **`eps`** (ε): the maximum distance between two samples to be considered neighbours. Calibrate using a *k-nearest-neighbour distance plot* (sort distances to the k-th nearest neighbour and look for an "elbow").
- **`min_samples`**: the minimum number of points in a neighbourhood to form a core point. A common starting heuristic is `min_samples` $= 2 \times d$ where d is the number of features.
- **Scaling**: `DBSCAN` is distance-based; always standardise features first.

General usage pattern:

```
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
import numpy as np

# k-NN distance plot to guide eps selection
nn = NearestNeighbors(n_neighbors=min_samples)
nn.fit(X_scaled)
distances, _ = nn.kneighbors(X_scaled)
knn_dist = np.sort(distances[:, -1])  # plot knn_dist vs index

dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels = dbscan.fit_predict(X_scaled)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise    = (labels == -1).sum()
```

Report the number of clusters found and the fraction of points labelled as noise; both quantities are informative about the density structure of the data.

Hierarchical (Agglomerative) Clustering

`Hierarchical clustering` builds a *dendrogram* by iteratively merging the closest pair of clusters (bottom-up / agglomerative). The full dendrogram reveals the cluster hierarchy at all scales; cutting at a chosen height produces a flat partition.

Key parameters:

- **`n_clusters`**: the number of clusters at which the dendrogram is cut. Use the dendrogram itself to identify a natural gap; compare with the `K-Means` elbow to check consistency.
- **`linkage`**: criterion for measuring inter-cluster distance. `'ward'` minimises intra-cluster variance and is the most common choice for continuous data; `'complete'` and `'average'` are useful comparisons.
- **`affinity / metric`**: `'euclidean'` (default for `ward`); `'cosine'` or `'manhattan'` for sparse/text-derived features.

General usage pattern:

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Plot dendrogram on a sample (full data may be too large)
Z = linkage(X_scaled[:500], method='ward')
plt.figure(figsize=(12, 5))
dendrogram(Z, truncate_mode='level', p=5)
plt.title('Hierarchical Clustering Dendrogram (sample)')
plt.savefig('dendrogram.png')

# Fit with chosen n_clusters
hc = AgglomerativeClustering(n_clusters=k, linkage='ward')
labels = hc.fit_predict(X_scaled)
```

Suggested workflow

1. **Data acquisition and EDA.** Download the chosen dataset from the assigned open-data source. Explore distributions, missing values, outlier rates, and feature correlations. Document data quality decisions.
2. **Preprocessing.** Handle missing values (imputation or removal). Encode categorical variables. Apply `StandardScaler` or `MinMaxScaler`. Optionally reduce dimensionality: apply `PCA` retaining $\geq 90\%$ variance before clustering to reduce noise and computation.
3. **Hyperparameter selection.**
   a. *`K-Means`*: plot inertia and Silhouette score for $k \in \{2, \ldots, 12\}$.
   b. *`DBSCAN`*: plot k-NN distances to identify ε; sweep `min_samples` $\in \{3, 5, 10, 2 \times d\}$.
   c. *`Hierarchical`*: inspect dendrogram; compare `ward`, `complete`, and `average` linkage.
4. **Clustering.** Fit all three algorithms on the same preprocessed feature matrix. Record labels, number of clusters found, and noise fractions.

5. **Evaluation.** Compute all three internal validation metrics for each algorithm configuration (see Section below). If ground-truth labels are available (e.g., from Challenge 2), also compute the Adjusted Rand Index as an external reference — but never use external labels during cluster optimisation.

6. **Visualisation.** Project the data to 2-D using `PCA` (and optionally `t-SNE` or `UMAP`) and colour-code each projection by cluster label. Produce one figure per algorithm showing the cluster structure.

7. **Interpretation.** Compute per-cluster feature means and describe each cluster in domain terms. What real-world phenomenon does each cluster represent? Do clusters align with known domain categories (e.g., urban vs. rural, peak vs. off-peak, high-risk vs. low-risk)?

Cluster evaluation metrics

Internal validation metrics — those that require only the data and cluster labels, not ground truth — must be reported for every algorithm configuration:

- **Silhouette Score** $\in [-1, 1]$: measures cohesion and separation. A score near 1 indicates well-separated, compact clusters; a score near 0 indicates overlapping clusters; negative values indicate misassignment.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}} \tag{1}$$

  where $a(i)$ is the mean intra-cluster distance and $b(i)$ is the mean nearest-cluster distance. Report the *mean* over all samples.

- **Davies–Bouldin Index** (lower is better): the average ratio of within-cluster scatter to between-cluster separation. Values close to 0 indicate compact, well-separated clusters.

- **Calinski–Harabasz Index** (higher is better): ratio of between-cluster dispersion to within-cluster dispersion across all clusters. Often called the *variance ratio criterion*.

- **Inertia** (`K-Means` only): sum of squared distances to centroids; used for the elbow plot but not comparable across algorithms.

- **Noise fraction** (`DBSCAN` only): proportion of points labelled -1. A high noise fraction may indicate that ε is too small or that the data has no meaningful dense structure.

- **Adjusted Rand Index (ARI)** (optional, when ground-truth labels exist): measures agreement between discovered clusters and known categories. Use only for post-hoc interpretation, never for selecting hyperparameters.

All metrics are available in `sklearn.metrics` (e.g., `silhouette_score`, `davies_bouldin_score`, `calinski_harabasz_score`).

Experimental design recommendations

- Repeat each configuration with at least **three independent random seeds** for algorithms with stochastic initialisation (`K-Means`); report mean and standard deviation of all metrics.
- Conduct at least one **dimensionality ablation**: compare clustering results on the full feature set vs. a `PCA`-reduced representation.
- Test **at least two feature subsets**: all features vs. a domain-motivated subset (e.g., only spatial features, only temporal features, only demographic features) and compare which yields more interpretable clusters.
- Provide a **summary comparison table** showing all algorithms side-by-side under their best configurations with Silhouette, Davies–Bouldin, and Calinski–Harabasz scores.
- Include a **dendrogram** for `Hierarchical clustering` (truncated if the dataset is large) and an **elbow + silhouette curve** for `K-Means`.

Common pitfalls (teaching moments)

- *Forgetting to scale*: `K-Means` and `DBSCAN` are distance-based; unscaled features will cause high-variance features to dominate cluster assignment.
- *Assuming `K-Means` is always better*: `K-Means` forces every point into a cluster and assumes convex shapes; `DBSCAN` and `Hierarchical` may reveal richer structure in non-spherical data.
- *Choosing k by lowest inertia*: inertia always decreases as k increases; use the elbow combined with Silhouette score, not inertia alone.
- *Ignoring `DBSCAN` noise*: a large noise fraction is data — not an error. Investigate noise points; they may represent genuine anomalies of domain interest.
- *Interpreting metrics without visualisation*: a high Silhouette score on a `PCA` projection that discards 40% of variance can be misleading. Cross-check metrics with 2-D visualisations.
- *Overfitting the number of clusters*: choosing k that merely separates individual outliers produces fragmented, uninterpretable clusters. Prefer solutions where each cluster has a coherent domain interpretation.
- *Using labels for tuning*: if Challenge 2 labels are available for the same data, they must never influence parameter selection. Use ARI only after the best configuration has been fixed by internal metrics.

Evaluation criteria

Submissions will be evaluated on the following aspects:

- **Implementation and correctness (20%):** all three algorithms are correctly applied with appropriate preprocessing and scaling; hyperparameter selection is documented and justified.
- **Experimental rigour (25%):** sensitivity analysis is performed, results are reported with uncertainty estimates (where applicable), and the comparison table is complete and fair.
- **Evaluation and metrics (20%):** all required internal metrics are computed and interpreted correctly; visualisations (elbow curves, dendrograms, 2-D projections) are included and well-labelled.
- **Interpretation and domain insight (25%):** discovered clusters are described in domain terms; the discussion addresses what the clustering reveals about the real-world dataset and which algorithm is best suited for this data and why.
- **Code quality, reproducibility, and video (10%):** clear instructions, fixed seeds, environment specification, and a short demonstration video.

Expectations and experimental protocol

Groups must follow a rigorous protocol. At minimum, the following elements are required:

1. Apply **identical preprocessing** to all three algorithms; the feature matrix fed to each clusterer must be the same.
2. Perform a documented **hyperparameter search** for each algorithm: elbow plot and Silhouette curve for `K-Means`; k-NN distance plot for `DBSCAN`; dendrogram inspection for `Hierarchical`. Justify the final chosen configuration.
3. Report all three internal metrics for the **best configuration of each algorithm** and include the comparison table in the paper.
4. Produce at least one **2-D scatter plot per algorithm** with cluster colour-coding (`PCA` projection or feature pair justified by EDA).
5. Provide a **cluster profile table**: for each discovered cluster, report the mean value of the most informative features and a one-sentence domain interpretation.
6. Document all **random seeds** used in `K-Means` initialisation and any randomised dimensionality reduction steps.

Deliverables

- A public **GitHub repository** named `challenge-5_<groupid>` containing:
  - *Reproducible code*: clear `Jupyter notebook(s)` or scripts reproducing all experiments; provide a single *best-run* notebook and separate utility modules if needed.
  - *Environment specification*: `requirements.txt` or `pyproject.toml`; a `Dockerfile` is strongly encouraged.
  - Exact commands or a `Makefile` to reproduce all reported results; record all random seeds.
  - *Figures*: elbow + Silhouette curves (`K-Means`), k-NN distance plot (`DBSCAN`), dendrogram (`Hierarchical`), and 2-D cluster visualisations for all algorithms saved as `.png` or `.pdf`.
  - *Results table*: a `.csv` or structured table of all metric values across algorithm configurations.
- A **scientific paper** in IEEE format, maximum **six (6) pages** excluding references. The paper must include: abstract, brief related work on clustering, data description (source, preprocessing decisions), methodology (algorithm and hyperparameter descriptions), experimental results (metrics table, figures), comparative discussion, domain interpretation of clusters, and conclusions.
- A **short video** (7–10 minutes) uploaded to a public or unlisted link (e.g., OneDrive) demonstrating the project, explaining the methodology and results, and including a brief live walkthrough of the best clustering result.

Reproducibility checklist

- Fix and record all `random_state` / `seed` values.
- Provide environment specification (`requirements.txt` or `pyproject.toml`). Docker is encouraged but optional.
- Include run scripts or `Makefile` instructions for ease of evaluation.
- Confirm that the same preprocessing pipeline is applied to the feature matrix fed to all three clustering algorithms.

Notes on scope and dataset size

Clustering large datasets (100 000+ records) with `Hierarchical clustering` or `DBSCAN` can be computationally expensive. When this is the case:

- Apply `Hierarchical clustering` to a stratified random sample of 5 000–10 000 records and document this constraint clearly.
- Use `MiniBatchKMeans` in `scikit-learn` for large `K-Means` experiments.

- For `DBSCAN` on large datasets, use `sklearn.cluster.OPTICS` (which scales better) or apply `DBSCAN` to the `PCA`-reduced 10–20 components.
- Report the dataset size and any subsampling decisions in the paper.

Grading checklist (to include in the repository)

Each submission must include a short `CHECKLIST.md` containing:

- The chosen dataset name, source URL, number of records, and number of features used after preprocessing.
- The final hyperparameter configuration for each of the three algorithms (e.g., k, `eps`, `min_samples`, `linkage`).
- The Silhouette, Davies–Bouldin, and Calinski–Harabasz scores for the best configuration of each algorithm.
- A short paragraph (max 200 words) comparing the three algorithms on this specific dataset and recommending which one is most appropriate and why.

References and further reading

Provide canonical references in the IEEE paper: the original `K-Means` paper (Lloyd, 1982; MacQueen, 1967), `DBSCAN` (Ester et al., 1996), and Ward linkage (Ward, 1963). Include at least one recent survey on clustering evaluation and validation metrics. Consult the `scikit-learn` clustering documentation and the dataset agency documentation for data provenance and licensing. All citations must be in IEEE style.

Challenge 5 shifts the learning paradigm from supervised signal to purely structural discovery. The pedagogical goal is not to optimise a single metric but to develop the judgment to distinguish meaningful structure from artefacts, and to communicate what clusters reveal about the world from which the data was collected.